v4.0 · May 2026

COGNITIVE COMPUTER SCIENCE · 7-MODEL RESEARCH STUDY

ADVERSARIAL COGNITION DIVERGENCE

Mapping Perceptual Breakdown Across Seven Neural Architectures and Human Vision using Adversarial Attacks and Psychophysics

What makes vision unbreakable — and do machines have it?

Is the robustness of human vision a matter of architecture, training, memory, or meaning — and can a machine ever have it?

BagNet	ResNet	EffNet	Shape-RN	CORnet-S	ViT	CLIP	Human
LOCAL	TEXTURE	SCALED	SHAPE	RECURRENT	GLOBAL	SEMANTIC	TRUTH

Mina (FerrariKazu)	Sandy	Youssef	Eyad	Mariam

Sadat Academy for Management Sciences · AI Department · Year 3

DEADLINE: Sunday, May 17, 2026

TABLE OF CONTENTS

1. PROJECT OVERVIEW AND EXPANDED RESEARCH QUESTION

Version 4.0 represents the full scientific maturity of this project. Three models are now fully complete and verified. Four models are in active development or pending final results. Most critically, this version introduces two architectures that test entirely new theories of robustness: CORnet-S (recurrent biological feedback loops) and CLIP ViT-B/32 (vision-language semantic grounding). Together with Shape-ResNet, these three additions mean the study now simultaneously tests three distinct scientific hypotheses about why human vision is adversarially robust in a way no machine has matched.

Component	v1.0	v2.0	v3.0	v4.0 (Current)
Models	ResNet-18	ResNet+EffNet+ViT+BagNet	5 models + Shape-ResNet	**7 models + Human**
Team	Solo (Mina)	Mina + Sandy + 2	Mina+Sandy+Youssef+Eyad	**Full team + Mariam**
Key finding	CNN vs human	Architecture spectrum	Arch + training objective	**3 theories of robustness**
New addition	Divergence curve	4-model spectrum	Shape-ResNet control	**CORnet-S + CLIP**
Verified complete	ResNet-18	ResNet + EfficientNet	ResNet+EffNet+ViT	**ResNet+EffNet+ViT+ShapeRN**

Central Research Question

What makes vision unbreakable — and do machines have it?

Is the robustness of human vision a matter of architecture, training, memory, or meaning — and can a machine ever have it?

The Three Hypotheses Under Test

Hypothesis	Model	Prediction	What a positive result means
Training objective drives robustness	Shape-ResNet vs ResNet-18	Shape-ResNet more robust despite identical architecture	Texture bias is the primary mechanism behind CNN fragility
Recurrent feedback drives robustness	CORnet-S vs feedforward CNNs	CORnet-S more robust than ResNet at high epsilon	Top-down feedback loops are what makes biological vision robust
Semantic grounding drives robustness	CLIP vs ViT	CLIP more robust than ViT despite similar architecture	Language-anchored representations resist adversarial pixel statistics

2. THE 7-MODEL ARCHITECTURE SPECTRUM

Seven models span the full computational spectrum from purely local patch-based processing to language-grounded semantic representation. Each model occupies a unique position in the spectrum and tests a different hypothesis about the origin of adversarial robustness.

#	Model	Owner	Core Property	Adversarial Prediction	Status
1	BagNet-33	Eyad	Pure local 33x33 patches	Collapses fastest — lower bound	PENDING
2	ResNet-18	Mina	Local conv, texture-biased	Collapses early — baseline CNN	COMPLETE
3	EfficientNet-B0	Mina	Compound-scaled CNN	Near-instant collapse at eps=0.01	COMPLETE
4	Shape-ResNet-50	Sandy	Shape-biased training (SIN)	More robust than ResNet — key test	COMPLETE
5	CORnet-S	Youssef +Eyad	Recurrent biological feedback	Better than feedforward CNNs	PENDING
6	ViT-Small	Mina	Global patch attention	Most robust among CNNs and ViTs	COMPLETE
7	CLIP ViT-B/32	Mariam	Vision-language contrastive	Most robust machine — semantic anchor	PENDING
8	Human	All	Shape + top-down feedback	Biological ground truth	IN PROGRESS

The Four Critical Comparisons v4.0 Enables

Comparison	What is isolated	Models compared	Positive result means
A	Training objective alone	Shape-ResNet vs ResNet-18	Shape-biased training confers robustness
B	Architecture alone	ViT-Small vs ResNet-18	Global attention confers robustness
C	Recurrence vs feedforward	CORnet-S vs ResNet-18	Biological feedback loops are key
D	Language grounding vs vision	CLIP vs ViT-Small	Semantic anchoring is the deepest protection

3. TEAM SPLIT AND RESPONSIBILITIES

Member	Model(s)	Key Branches	Deliverables
Mina (FerrariKazu)	ResNet-18 EfficientNet-B0 ViT-Small + Full Pipeline	phase/1-resnet phase/1-efficientnet phase/1-vit phase/4-analysis phase/5-sdt	Full pipeline ownership. ResNet DONE (95.82%). EfficientNet DONE (96.81%, BIM). ViT DONE (97.80%). Human study form. Phase 4 + 5 scripts. 7-model divergence figures.
Sandy	Shape-ResNet-50	phase/1-shaperesnet phase/2-shaperesnet docs/report	Shape-ResNet DONE (91.47% clean, adversarial verified). Attack arrays generated. Per-class heatmap. Final report theoretical section. Presentation slides.
Youssef + Eyad	CORnet-S	phase/1-cornets phase/2-cornets	CORnet-S full setup from scratch (see Section 6.5). Training on CIFAR-10. Attack generation. Accuracy table. Figures committed to phase4_analysis/figures/cornets/
Eyad	BagNet-33 + Texture Analysis	phase/1-bagnet phase/2-bagnet	BagNet training (target: 75-82%). Attack generation. Texture analysis of BagNet vs human responses. Accuracy table. Figures to phase4_analysis/figures/bagnet/
Mariam	CLIP ViT-B/32	phase/1-clip phase/2-clip	Full WSL2 setup from scratch (see Section 5.2). CLIP zero-shot evaluation on CIFAR-10. Adversarial robustness testing. Zero-shot prompt specification. Full setup in Section 6.6.

4. REPOSITORY STRUCTURE

github.com/FerrariKazu/Adversarial-Cognitive-Model

```
Adversarial-Cognitive-Model/
README.md
requirements.txt
.gitignore
config/
train_config.yaml # Shared training hyperparams
attack_config.yaml # Epsilon levels, attack params
experiment_config.yaml # Human study params
phase1_training/
model_resnet.py # ResNet-18 wrapper [Mina - DONE]
model_vit.py # ViT-Small wrapper [Mina - DONE]
model_efficientnet.py # EfficientNet-B0 [Mina - DONE]
model_shaperesnet.py # Shape-ResNet-50 [Sandy - DONE]
model_cornets.py # CORnet-S [Youssef+Eyad]
model_clip.py # CLIP ViT-B/32 [Mariam]
model_bagnet.py # BagNet-33 [Eyad]
dataset.py # Shared CIFAR-10 loader
train.py # Generic training loop
evaluate.py # Shared evaluation script
checkpoints/ # .pth files (gitignored)
phase2_attacks/
fgsm.py # DONE - with cifar_min/max clamping
pgd.py # DONE - BIM mode for noise-sensitive models
cw.py # DONE
generate_adv_all_models.py # All 7 models, conditional BIM toggle
adv_images/ # .npy arrays (gitignored)
resnet/ # DONE - 14 files
vit/ # DONE
efficientnet/ # DONE
shaperesnet/ # DONE
cornets/ # Pending
clip/ # Pending
bagnet/ # Pending
phase3_human_study/ # DONE
phase4_analysis/
divergence_curves.py # 7-model + human accuracy vs epsilon
class_heatmap.py # Per-class vulnerability all models
gradcam.py # Grad-CAM (ResNet + Shape-ResNet only)
cross_model_compare.py # Side-by-side model comparison
figures/ # All output figures
phase5_sdt/
```

```
sdt_core.py
sdt_analysis.py # d-prime for all 7 models + human
sdt_plots.py
utils/
metrics.py
visualize.py
logger.py
```

5. WSL2 ENVIRONMENT SETUP

5.1 Existing Members (Mina, Sandy, Youssef, Eyad)

If your environment is already running and you have trained at least one model, your CUDA, venv, and PyTorch installation are fine. You only need to install the two new libraries below.

```
# Activate your existing venv first
source ~/Adversarial-Cognitive-Model/.venv/bin/activate

# For CORnet-S (Youssef and Eyad)
pip install git+https://github.com/dicarlolab/CORnet.git

# Verify CORnet-S loads
python3 -c "import cornet; m = cornet.cornet_s(); print('CORnet-S OK')"
```

5.2 Mariam — Full WSL2 Setup from Scratch

Mariam needs a complete environment from zero. Follow every step in order. Do not skip any step. If any command fails, stop and fix it before continuing.

NOTE: CLIP is a heavy model. You will need at least 8 GB of free disk space and a stable internet connection.

Step 1 — Enable WSL2 on Windows

```
# Open PowerShell as Administrator (right-click -> Run as administrator)
# Paste and run this single command:
wsl --install

# This installs WSL2 and Ubuntu automatically.
# Restart your computer when prompted.
# After restart, Ubuntu will open and ask you to create a username and password.
# Choose a simple username (e.g. mariam) and remember your password.
```

Step 2 — Update Ubuntu and Install System Dependencies

```
# Run these commands inside the Ubuntu terminal:
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv git wget build-essential
sudo apt install -y libgl1-mesa-glx libglib2.0-0
```

Step 3 — Clone the Project Repository

```
cd ~
git clone https://github.com/FerrariKazu/Adversarial-Cognitive-Model.git
cd Adversarial-Cognitive-Model
```

Step 4 — Create and Activate a Python Virtual Environment

```
python3 -m venv .venv
source .venv/bin/activate

# You should now see (.venv) at the start of your terminal line.
# Every time you open a new terminal, run the source command above first.
```

Step 5 — Install PyTorch and Core Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt

# If requirements.txt fails on any package, install PyTorch manually:
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu

# Note: CPU-only install is fine for CLIP zero-shot evaluation.
# If you have an NVIDIA GPU, ask Mina for the GPU-specific install command.
```

Step 6 — Install CLIP

```
pip install git+https://github.com/openai/CLIP.git
pip install ftfy regex tqdm

# Verify CLIP installed correctly:
python3 -c "import clip; print(clip.available_models())"
# Expected output: ['RN50', 'RN101', ..., 'ViT-B/32', ...]
```

Step 7 — Create Your Branch

```
git checkout -b phase/1-clip

# You are now on your own branch. All your work goes here.
# When done: git add . && git commit -m 'feat: CLIP evaluation complete'
# Then: git push origin phase/1-clip
# Then open a Pull Request on GitHub pointing to the dev branch.
```

6. PHASE 1 — MODEL TRAINING (ALL 7 MODELS)

6.1 ResNet-18 [Mina — COMPLETE]

Clean accuracy: 95.82% at epoch 44	**All 14 adversarial .npy arrays generated and verified**	**Pixel-space clamping fix applied**

No further action needed.

6.2 EfficientNet-B0 [Mina — COMPLETE]

Clean accuracy: 96.81%	**BIM mode (no random start) — monotonic collapse verified**	**Collapses to 0.93% at eps=0.01**

EfficientNet required BIM (Basic Iterative Method) instead of PGD with random start. High-epsilon random initialization at 224x224 caused gradient instability in Swish and BatchNorm layers, producing a false accuracy 'rebound'. Inner-loop clamping using cifar_min/cifar_max bounds and disabling random initialization resolved this completely. The collapse at eps=0.01 (96.81% to 0.93%) is one of the most striking findings in the study.

6.3 ViT-Small [Mina — COMPLETE]

Clean accuracy: 97.80%	**Most robust machine model in the study**	**Retains 0.58% at eps=0.30**

No further action needed.

6.4 Shape-ResNet-50 [Sandy — COMPLETE]

Clean accuracy: 91.47% (fixed — normalization mismatch resolved)	**Fine-tuned 15 epochs, reached 91.48% during training**	**Adversarial arrays regenerated after normalization fix**

Shape-ResNet is a ResNet-50 whose weights were pretrained on Stylized-ImageNet (SIN) by Geirhos et al. (2019). SIN replaces all image textures with artistic styles, forcing the network to learn shape rather than texture. Sandy downloads these weights and fine-tunes only the final FC layer on CIFAR-10. The scientific importance is that this model has the IDENTICAL architecture to ResNet-18 but a fundamentally different representational bias.

> *NOTE: The earlier reported 48.74% clean accuracy was a normalization mismatch bug. The correct clean accuracy after fix is 91.47%. All adversarial results below use the corrected evaluation pipeline.*

6.5 CORnet-S [Youssef + Eyad — PENDING]

CORnet-S is a recurrent neural network explicitly designed to model the primate visual cortex. It contains four stages (V1, V2, V4, IT) with recurrent connections in each stage. Unlike all other models in this study, CORnet-S processes each image through multiple feedback passes, mimicking how the human brain refines its visual representation over time. If CORnet-S is more adversarially robust than the feedforward CNNs, this constitutes direct evidence that biological feedback loops are what protect human vision.

> *Youssef is the lead. Eyad assists. Both work on the same branch: phase/1-cornets.*

Installation

```
# Activate your venv first
source ~/Adversarial-Cognitive-Model/.venv/bin/activate

# Install CORnet from the DiCarlo Lab official repository
pip install git+https://github.com/dicarlolab/CORnet.git

# Verify it loads correctly
python3 -c "import cornet; m = cornet.cornet_s(); print(m)"
# If this prints the model architecture, installation succeeded.
```

Model Code

```
# phase1_training/model_cornets.py
import torch
import torch.nn as nn
import cornet

class CIFARCORnet(nn.Module):
"""
CORnet-S: recurrent model of the primate visual ventral stream.
Four areas: V1 -> V2 -> V4 -> IT, each with recurrent feedback.
This is the ONLY model in our study with recurrent connections.
If more robust than ResNet: feedback loops drive adversarial robustness.
"""
def __init__(self, num_classes=10, times=5):
super().__init__()
# Load pretrained CORnet-S (ImageNet weights from DiCarlo Lab)
self.model = cornet.cornet_s(pretrained=True, map_location='cpu')
# Replace the decoder (final linear layer) for CIFAR-10
self.model.decoder.linear = nn.Linear(512, num_classes)
# times controls how many recurrent iterations per forward pass
self.times = times

def forward(self, x):
return self.model(x)
```

Training

```
# CORnet-S expects 224x224 input. Add Resize(224) to dataloader.
# Edit dataset.py to include transforms.Resize(224) for CORnet loader.

# Training command:
cd ~/Adversarial-Cognitive-Model/phase1_training
python train.py --model cornets --config ../config/train_config.yaml

# Expected training time: 90-120 min on RTX 4060 (recurrence = slower)
# Target clean accuracy: 75-88% on CIFAR-10
# Note: lower than ViT is expected. Recurrence is the contribution, not accuracy.
# Check papers: CORnet-S achieves ~80-85% on CIFAR-10 with standard fine-tuning.
```

Expected Adversarial Behavior

CORnet-S should be more robust than ResNet-18 and EfficientNet at intermediate epsilon values (0.05 to 0.20) due to its recurrent feedback mechanism. The recurrent passes allow the model to iteratively refine its representation, potentially correcting for the small pixel-level perturbations that fool feedforward models. Whether it exceeds Shape-ResNet is the key scientific comparison.

6.6 CLIP ViT-B/32 [Mariam — PENDING]

CLIP (Contrastive Language-Image Pretraining) was trained by OpenAI on 400 million image-text pairs from the internet. It was NOT trained to classify images. Instead, it learned to align visual representations with language descriptions. When we evaluate CLIP on CIFAR-10, we use zero-shot classification: we give it the image and the text prompt for each class (e.g. 'a photo of a cat') and it scores which text best matches the image. This is fundamentally different from every other model in the study.

The scientific prediction: adversarial perturbations are designed to fool visual pixel statistics. CLIP's representations are anchored to language meaning, not pixel statistics. Therefore CLIP should be the most adversarially robust machine model in the study, sitting between ViT and Human on the robustness spectrum.

IMPORTANT: Mariam must complete the full WSL2 setup in Section 5.2 before starting anything in this section.

Model Code

```
# phase1_training/model_clip.py
import torch
import clip
from torchvision import transforms

# These are the STANDARDISED text prompts for all 10 CIFAR-10 classes.
# Do NOT change these prompts. They must be identical across all experiments.
CIFAR10_PROMPTS = [
'a photo of an airplane',
'a photo of an automobile',
'a photo of a bird',
'a photo of a cat',
'a photo of a deer',
'a photo of a dog',
'a photo of a frog',
'a photo of a horse',
'a photo of a ship',
'a photo of a truck',
]

class CIFARClip:
"""
CLIP ViT-B/32 zero-shot classifier for CIFAR-10.
No training required. Uses contrastive image-text similarity.
The model scores how well each text prompt matches the image.
Highest scoring prompt = predicted class.
"""
def __init__(self, device='cuda' if torch.cuda.is_available() else 'cpu'):
self.device = device
# Load CLIP ViT-B/32 (downloads ~350MB on first run, be patient)
self.model, self.preprocess = clip.load('ViT-B/32', device=device)
self.model.eval()
# Tokenize all 10 class prompts once (reused for every batch)
text_tokens = clip.tokenize(CIFAR10_PROMPTS).to(device)
with torch.no_grad():
```

```
        self.text_features = self.model.encode_text(text_tokens)
        self.text_features = self.text_features / self.text_features.norm(
        dim=-1, keepdim=True
        )

    def predict(self, images):
        # images: tensor of shape (B, 3, H, W), values in [0, 1]
        # Resize to 224x224 (CLIP's expected input size)
        images = torch.nn.functional.interpolate(
        images, size=(224, 224), mode='bilinear', align_corners=False
        )
        with torch.no_grad():
            image_features = self.model.encode_image(images)
            image_features = image_features / image_features.norm(
            dim=-1, keepdim=True
            )
            # Cosine similarity between image and all 10 text prompts
            logits = (100.0 * image_features @ self.text_features.T)
            return logits # shape: (B, 10)
```

Evaluation Script

```
# phase1_training/evaluate_clip.py
# Run this to get CLIP clean accuracy on CIFAR-10 test set
import torch
from torchvision import datasets, transforms
from model_clip import CIFARClip

device = 'cuda' if torch.cuda.is_available() else 'cpu'
clip_model = CIFARClip(device=device)

test_data = datasets.CIFAR10(
root='./data', train=False, download=True,
transform=transforms.ToTensor() # CLIP handles its own preprocessing
)
loader = torch.utils.data.DataLoader(test_data, batch_size=64, shuffle=False)

correct = 0
total = 0
for images, labels in loader:
images, labels = images.to(device), labels.to(device)
logits = clip_model.predict(images)
preds = logits.argmax(dim=1)
correct += (preds == labels).sum().item()
total += labels.size(0)

print(f'CLIP ViT-B/32 zero-shot accuracy: {100*correct/total:.2f}%')
# Expected: 65-75% (zero-shot on CIFAR-10 without any fine-tuning)
```

Running CLIP Adversarial Evaluation

```
# After clean accuracy is confirmed, run adversarial attacks on CLIP.
# CLIP is a zero-shot model so we attack the image encoder, not a classifier head.
python phase2_attacks/generate_adv_all_models.py --model clip
```

```
# Then evaluate:
python phase1_training/evaluate_clip_adv.py

# IMPORTANT: CLIP may require special handling in generate_adv_all_models.py
# because its forward pass returns logits differently.
# If you get an error, paste it into Claude Code with this message:
# 'CLIP adversarial generation failed with this error: [error text].'
# 'Our model uses zero-shot contrastive similarity. Fix the attack script.'
```

7. PHASE 2 — ADVERSARIAL ATTACK GENERATION

Complete and verified for ResNet-18, EfficientNet-B0, ViT-Small, and Shape-ResNet.

7.1 Pipeline Architecture (Updated for v4)

The attack pipeline uses PGD with inner-loop clamping for all models. EfficientNet uses BIM (PGD without random initialization) due to gradient instability in Swish/BatchNorm layers at 224x224 resolution. CLIP requires special handling due to its zero-shot nature and is treated as a special case in the generation script.

```
# generate_adv_all_models.py — MODEL_CONFIGS (updated for v4)
MODEL_CONFIGS = {
'resnet': {'ckpt': 'checkpoints/resnet18_best.pth',
'class': 'CIFARResNet', 'out': 'adv_images/resnet/'},
'efficientnet':{'ckpt': 'checkpoints/efficientnet_best.pth',
'class': 'CIFAREfficientNet','out': 'adv_images/efficientnet/',
'bim': True}, # BIM mode — no random start
'vit': {'ckpt': 'checkpoints/vit_small_best.pth',
'class': 'CIFARViT', 'out': 'adv_images/vit/'},
'shaperesnet': {'ckpt': 'checkpoints/shaperesnet50_best.pth',
'class': 'ShapeResNet', 'out': 'adv_images/shaperesnet/'},
'cornets': {'ckpt': 'checkpoints/cornets_best.pth',
'class': 'CIFARCORnet', 'out': 'adv_images/cornets/'},
'clip': {'class': 'CIFARClip', 'out': 'adv_images/clip/',
'zero_shot': True}, # No checkpoint needed
'bagnet': {'ckpt': 'checkpoints/bagnet33_best.pth',
'class': 'CIFARBagNet', 'out': 'adv_images/bagnet/'},
}
# Run command per member:
python generate_adv_all_models.py --model cornets # Youssef + Eyad
python generate_adv_all_models.py --model clip # Mariam
python generate_adv_all_models.py --model bagnet # Eyad
```

7.2 Adversarial Accuracy Table (Current + Predicted)

Epsilon	BagNet-33	ResNet-18	EffNet-B0	Shape-RN50	CORnet-S	ViT-Small	CLIP	Human
0.00 (Clean)	~80%*	**95.82%**	**96.81%**	**91.47%**	~83%*	97.80%	~70%*	74.25%
0.01	~5%*	**75.59%**	**0.93%**	**18.11%**	~35%*	55.18%	~55%*	62.75%
0.05	~0%*	**2.82%**	**0.00%**	**0.01%**	~8%*	8.56%	~20%*	69.00%
0.10	~0%*	**0.21%**	**0.00%**	**0.00%**	~3%*	2.93%	~10%*	59.25%
0.20	~0%*	**0.01%**	**0.00%**	**0.00%**	~1%*	1.42%	~5%*	64.25%
0.30	~0%*	**0.00%**	**0.00%**	**0.00%**	~0%*	0.58%	~3%*	60.25%

* Predicted values from research literature. Will be replaced with measured results.

Bold = verified measured values.

8. PHASE 3 — HUMAN PSYCHOPHYSICS EXPERIMENT

All scripts built and verified. Google Form in progress. Target: 15 participants minimum.

The human study is the biological ground truth against which all seven models are benchmarked. Participants view PGD-perturbed CIFAR-10 images at six epsilon levels and classify them. Their accuracy is then plotted alongside all seven models on the same divergence curve, showing exactly where each machine system departs from human-level robustness.

Team Member	Target Participants	Method
Mina	5	Classmates, AI department colleagues
Sandy	5	Social circle, university contacts
Youssef	3	Fill form + 2 recruits
Eyad	3	Fill form + 2 recruits
Mariam	3	Fill form + 2 recruits (ADDED in v4)
TOTAL	**19**	**Above minimum threshold of 15**

9. PHASE 4 — 7-MODEL DIVERGENCE ANALYSIS

9.1 The Headline Figure

The headline figure is a single plot with 8 curves: all seven models plus humans, accuracy vs epsilon. This is the most comprehensive adversarial psychophysics plot ever produced at undergraduate level for CIFAR-10. Every other figure in the study supports this one.

```
# phase4_analysis/cross_model_compare.py
MODELS = {
'BagNet-33': {'color': '#F97316', 'adv_dir': 'adv_images/bagnet/'},
'ResNet-18': {'color': '#F85149', 'adv_dir': 'adv_images/resnet/'},
'EfficientNet-B0': {'color': '#58A6FF', 'adv_dir': 'adv_images/efficientnet/'},
'Shape-ResNet-50': {'color': '#3FB950', 'adv_dir': 'adv_images/shaperesnet/'},
'CORnet-S': {'color': '#BC8CFF', 'adv_dir': 'adv_images/cornets/'},
'ViT-Small': {'color': '#56D4DD', 'adv_dir': 'adv_images/vit/'},
'CLIP ViT-B/32': {'color': '#FF79C6', 'adv_dir': 'adv_images/clip/'},
}
# Human plotted as yellow dashed line at top
# Title: 'Adversarial Robustness: 7 Architectures vs. Human Vision'
```

9.2 All Phase 4 Figures

Figure	Script	Description	Owner
7model_divergence_curve.png	cross_model_compare.py	8 curves (7 models + humans) — HEADLINE	Mina
resnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability ResNet	Mina
efficientnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability EfficientNet	Mina
shaperesnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability Shape-ResNet	Sandy
cornets_class_heatmap.png	class_heatmap.py	Per-class vulnerability CORnet-S	Youssef
vit_class_heatmap.png	class_heatmap.py	Per-class vulnerability ViT	Mina
clip_class_heatmap.png	class_heatmap.py	Per-class vulnerability CLIP	Mariam
bagnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability BagNet	Eyad
training_vs_arch.png	cross_model_compare.py	ResNet vs Shape-ResNet isolation	Mina
recurrence_compare.png	cross_model_compare.py	CORnet-S vs feedforward CNNs	Mina
language_vs_vision.png	cross_model_compare.py	CLIP vs ViT-Small isolation	Mina

10. PHASE 5 — SIGNAL DETECTION THEORY (SDT)

SDT analysis computes d-prime for every system at every epsilon level, producing a single quantitative threshold: the epsilon at which each system drops below d'=1.0 (the boundary of meaningful discrimination). This threshold is the headline quantitative finding of the entire study.

System	d' at eps=0.00	d' at eps=0.05	d' at eps=0.10	d' < 1.0 at eps
Human	4.5 (near perfect)	4.2	3.8	~0.35 (predicted)
CLIP ViT-B/32	~3.5	~2.8	~1.8	~0.18 (predicted)
ViT-Small	~3.8	~2.1	~1.2	~0.12 (predicted)
CORnet-S	~3.2	~1.9	~1.0	~0.10 (predicted)
Shape-ResNet-50	~3.4 (measured)	~0.1	~0.0	~0.02 (measured)
ResNet-18	3.5 (measured)	~0.4	~0.1	~0.04 (measured)
EfficientNet-B0	~3.6 (measured)	~0.0	~0.0	~0.01 (measured)
BagNet-33	~2.8	~0.2	~0.0	~0.02 (predicted)

The Headline SDT Finding (v4)

Expected d' threshold order: BagNet < EfficientNet < Shape-ResNet < ResNet < CORnet-S < ViT < CLIP < Human. If CLIP d' threshold > ViT d' threshold: language grounding provides robustness beyond architecture alone. If CORnet-S d' threshold > ResNet d' threshold: biological recurrence provides robustness. If Shape-ResNet d' threshold > ResNet d' threshold: training objective drives robustness. Any or all three of these results are independently publishable. All three together constitute the most comprehensive adversarial robustness comparison at undergraduate level.

11. GITHUB COLLABORATION WORKFLOW

11.1 Branch Strategy

Branch	Owner	Status	Purpose
main	Mina	Protected	Merged releases only — PR required
dev	All	Active	Integration branch
phase/1-resnet	Mina	DONE	ResNet training — merged
phase/1-efficientnet	Mina	DONE	EfficientNet — merged (BIM fix)
phase/1-vit	Mina	DONE	ViT training — merged
phase/1-shaperesnet	Sandy	DONE	Shape-ResNet — normalization fixed
phase/1-cornets	Youssef+Eyad	PENDING	CORnet-S training
phase/1-clip	Mariam	PENDING	CLIP zero-shot evaluation
phase/1-bagnet	Eyad	PENDING	BagNet training
phase/2-attacks	Mina	DONE	ResNet attack scripts — merged
phase/2-cornets-atk	Youssef+Eyad	PENDING	CORnet-S adversarial arrays
phase/2-clip-atk	Mariam	PENDING	CLIP adversarial arrays
phase/2-bag-attacks	Eyad	PENDING	BagNet adversarial arrays
phase/3-human-study	Mina	DONE	Human study scripts — merged
phase/4-analysis	Mina	PENDING	7-model cross-model analysis
phase/5-sdt	Mina	PENDING	SDT d-prime all models
docs/report	Sandy	PENDING	Final report + slides

11.2 Standard Commit Workflow (All Members)

```
# After completing your model training and attack generation:
git add .
git commit -m 'feat: [modelname] training + attack generation complete'
git push origin phase/1-[yourmodel]

# Then open a Pull Request on GitHub:
# Base: dev Compare: phase/1-[yourmodel]
# Title: '[ModelName] Phase 1+2 Complete'
# Add Mina as reviewer.

# DO NOT push directly to main or dev.
# DO NOT commit .pth checkpoint files (they are in .gitignore).
# DO commit figures (.png) and accuracy tables (.csv).
```

12. DELIVERABLES AND SUBMISSION CHECKLIST

DEADLINE: Sunday, May 17, 2026

#	Deliverable	Owner	Status
1	ResNet-18 checkpoint (95.82%)	Mina	DONE
2	ResNet adversarial arrays (14 files)	Mina	DONE
3	EfficientNet-B0 checkpoint (96.81%, BIM)	Mina	DONE
4	EfficientNet adversarial arrays	Mina	DONE
5	ViT-Small checkpoint (97.80%)	Mina	DONE
6	ViT adversarial arrays	Mina	DONE
7	Shape-ResNet-50 checkpoint (91.47%, fixed)	Sandy	DONE
8	Shape-ResNet adversarial arrays (fixed)	Sandy	DONE
9	BagNet-33 checkpoint	Eyad	PENDING
10	BagNet adversarial arrays	Eyad	PENDING
11	CORnet-S checkpoint	Youssef+Eyad	PENDING
12	CORnet-S adversarial arrays	Youssef+Eyad	PENDING
13	CLIP ViT-B/32 zero-shot evaluation	Mariam	PENDING
14	CLIP adversarial arrays	Mariam	PENDING
15	Human study Google Form (live)	Mina	IN PROGRESS
16	Anonymized human responses (n >= 15)	All	PENDING
17	7-model + human divergence curve figure	Mina	PENDING
18	Training vs architecture isolation figure	Mina	PENDING
19	Recurrence vs feedforward figure	Mina	PENDING
20	Language vs vision isolation figure	Mina	PENDING
21	Per-class heatmaps (all 7 models)	All	PENDING
22	SDT d-prime curves (all 7 models)	Mina	PENDING
23	Final report with interpretation	Sandy	PENDING
24	Presentation slides	Sandy	PENDING
25	GitHub — all branches merged to main	Mina	PENDING

13. THEORETICAL FRAMEWORK AND EXPECTED RESULTS

13.1 Four Pillars of This Study

Theory	Source	What v4 tests specifically
Feedforward vs Feedback (DiCarlo & Cox, 2007)	Nature Neurosci.	CORnet-S is the only recurrent model. If it outperforms all feedforward models, top-down feedback is the mechanism behind biological robustness.
Texture Bias (Geirhos et al., 2019)	ICLR 2019	Shape-ResNet is Geirhos's own model. If it exceeds ResNet-18 in robustness, texture bias is the primary mechanism underlying CNN fragility.
Vision-Language Alignment (Radford et al., 2021)	OpenAI CLIP	CLIP is the only language-grounded model. If it exceeds ViT in robustness, semantic grounding in language provides a form of protection no purely visual training achieves.
Signal Detection Theory (Green & Swets, 1966)	Psychophysics	d-prime across all 7 models + humans. The epsilon where each system drops below d'=1.0 is the single most important quantitative finding.

13.2 Results Interpretation Guide

Outcome	Interpretation	Quote to use in report
Shape-ResNet > ResNet-18 (most likely)	Training objective is the key variable. Architecture alone is insufficient.	'Shape-biased training via Stylized-ImageNet confers adversarial robustness independent of architecture, implicating texture bias as the primary mechanism '
CORnet-S > ResNet-18 (expected)	Recurrent feedback confers adversarial robustness beyond feedforward processing.	'Biological recurrence, rather than architectural scale or depth alone, provides resistance to adversarial perturbation consistent with the DiCarlo feedback hypothesis'
CLIP > ViT-Small (expected)	Language grounding provides the deepest protection of any machine learning strategy tested.	'Vision-language contrastive training produces representations that are fundamentally more robust to adversarial pixel statistics than any purely visual training objective'
Any model approaches human d' threshold	Major finding. Would be the closest any CIFAR-10 model has come to human robustness.	Report exactly which model and at which epsilon the gap closes, if at all.

13.3 Key References

[1] Goodfellow et al. (2015) Explaining and Harnessing Adversarial Examples. ICLR 2015.

[2] Madry et al. (2018) Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR 2018.

[3] Carlini & Wagner (2017) Towards Evaluating the Robustness of Neural Networks. IEEE S&P; 2017.

[4] Geirhos et al. (2019) ImageNet-trained CNNs are biased towards texture. ICLR 2019. [Shape-ResNet source]

[5] Brendel & Bethge (2019) Approximating CNNs with Bag-of-local-Features models. ICLR 2019. [BagNet source]

[6] Dosovitskiy et al. (2021) An Image is Worth 16x16 Words: Transformers for Vision. ICLR 2021. [ViT source]

[7] Tan & Le (2019) EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.

[8] He et al. (2016) Deep Residual Learning for Image Recognition. CVPR 2016.

[9] Kubilius et al. (2019) Brain-Like Object Recognition with High-Performing Shallow RNNs. NeurIPS 2019. [CORnet-S source]

[10] Radford et al. (2021) Learning Transferable Visual Models from Natural Language Supervision. ICML 2021. [CLIP source]

[11] DiCarlo & Cox (2007) Untangling invariant object recognition. Trends in Cognitive Sciences.

[12] Yamins & DiCarlo (2016) Using goal-driven DNN models to understand sensory cortex. Nature Neuroscience.

[13] Green & Swets (1966) Signal Detection Theory and Psychophysics. Wiley.

[14] Selvaraju et al. (2017) Grad-CAM: Visual Explanations from Deep Networks. ICCV 2017.

Adversarial Cognition Divergence v4.0 | 7-Model Study | Cognitive Computer Science | Sadat Academy for Management Sciences | Mina (FerrariKazu), Sandy, Youssef, Eyad, Mariam | github.com/FerrariKazu/Adversarial-Cognitive-Model | May 2026